Promo Finder: ranked promos from a microdrama screenplay

Kuku AI Generalist assignment · Problem 1 (Find the Promos) · prototype: Python repo with run steps in `README.md`

1. Problem and why I picked it

An editor today watches every episode and picks 20–60 s promo moments by hand, one show at a time, while new shows ship every week. A good promo opens on a hook in its first 3 seconds, ends on an unresolved cliffhanger, and makes sense to a stranger. I chose this problem because its output can be checked: every promo is an episode plus start and end timestamps that an editor can open and judge in seconds.

2. Assumptions (where the brief was silent)

- The input is the screenplay text only: no video or audio. Script timestamps are assumed close enough to the video for an editor to fine-tune.
- The tool **proposes** and an editor **decides**. It ranks a shortlist of 30 promos with reasons and flags, not a single "best" clip.
- A promo has 1–3 segments (same or adjacent episodes), 20–60 s in total. For variety: at most 2 per episode, 1 per scene, 5 per block of 5 episodes.
- Free tiers only, and the model must be swappable without code changes, as the brief asks.

3. Solution and core workflow

One command (`make run SHOW_DIR=…`) runs five stages and writes `promos.json`, `promos.csv`, `ranking.md` and `quality_report.md`. A free rules-only mode (`make run-rules`) needs no API key. A local viewer (`make ui`) shows the ranking, cut lists and the report.

1. **Parse:** each episode file becomes a timeline of timestamped beats (scene, dialogue, action). Malformed lines produce warnings, not crashes.
2. **Generate candidates (code):** every 20–60 s window that starts and ends on a beat boundary (never mid-line) is a candidate (39,354 for this pack). Rule signals score the hook (first 3 s only), the cliffhanger and intensity from a keyword lexicon (slaps, reveals, threats, ending cues). The tool removes overlapping windows (non-maximum suppression, overlap > 0.5), drops "previously on" recaps and builds a few stitched hook-plus-cliffhanger pairs. The top 8 per episode plus 40 stitched candidates go to the LLM.
3. **Score (LLM, cascaded):** batches of 8 candidates are scored 0–10 on hook, cliffhanger, intensity and spoiler risk against fixed anchors. The model answers only with *beat IDs* from the clip, never free-text timestamps, so it cannot invent a cut. The top 60 then get a *blind* clarity check: the model sees only the clip, as a stranger would. It can also pick a stronger unresolved end point, which is re-validated by code.
4. **Rank (code + LLM):** rule and LLM scores are blended, hard gates are applied, and caps and de-duplication enforce variety. A listwise LLM rerank of the top 30 runs twice, the second time with the order reversed to cancel position bias.
5. **Evaluate (code):** the quality report covers validation counts, score distributions, LLM-vs-rule disagreement, a rules-only baseline, two ablations, failure modes with real examples, and tokens.

4. Architecture, tools, and what is AI vs code

Part	Done by	Implementation
Parsing, candidate search, rule signals, caps, validation of every cut	Code	Python 3.12, Pydantic schemas; YAML config and keyword lexicon (`resources/config/`). A cut that breaks a rule is never delivered.
Hook, cliffhanger, intensity, spoiler, blind clarity, end choice, rerank	LLM	Versioned prompts in `resources/prompts/`; JSON answers validated, then retried on failure
Provider layer	Code	Gemini, Groq, Anthropic, OpenAI, Ollama, mock or none via `.env`; response cache (resume without re-billing), per-minute/day rate limits, call and USD budgets.
Final choice of what to post	Editor	FastAPI + React editor and CSV; source excerpts, cut controls, saved verdicts and notes

Tested live: Gemini only (`gemini-3.5-flash-lite` for the full run and smoke runs, `gemini-3.1-flash-lite` in one smoke run). Groq, Anthropic, OpenAI and Ollama are tested only against mocked HTTP replies. Swapping models is a two-line `.env` change (`LLM_PROVIDER`, `LLM_MODEL`).

5. Identification and ranking method

```
dim   = 0.7·LLM + 0.3·rule            (clarity: 0.85·LLM + 0.15·rule)
score = 0.3·hook + 0.3·cliffhanger + 0.2·clarity + 0.2·intensity − 0.05·spoiler_risk
final = 0.5·score + 0.5·listwise rerank score        (top 30 only; stitched promos discounted by coherence)
```

- **Hard gates:** a promo is never delivered if the LLM scores its hook or its delivered ending below 5/10. The gates removed 117 of 428 candidates after the first pass and 2 more after blind scoring.
- **Confidence** rises when rules and LLM agree; flags (*needs context*, *weak ending*, *dead air*, *spoiler*) tell the editor what to check.

6. How the output answers the brief

Brief asks for	Where it is
Promos across all episodes, with the episode and start/end of every segment, in order	`segments[]` in `promos.json` / `.csv`; 30 promos drawn from 27 of the 50 episodes
The hook, the cliffhanger, one line on why it works	`hook`, `cliffhanger` (the beat, timestamp and line), `why`
Rank them: which to post first and why	`rank`, `post_first_because`, `ranking.md`; top 3 below
Check quality: where does it pick badly?	`quality_report.md` §9 and section 8 below

Top 3 to post first (run `20261108-153346`; scores 0–10):

#	Cut	Moment	Hook / cliff / alone
1	Ep 32 01:59.60–02:31.20 (32 s)	Public confrontation that ends on a weapon threat	7.5 / 7.5 / 7.5
2	Ep 41 00:57.80–01:33.50 (36 s)	Defiance in a chapel, cut on an explosion and an armed arrival	7.5 / 7.5 / 4.8
3	Ep 18 00:56.20–01:29.50 (33 s)	Fire rescue, cut on a falling beam	7.4 / 6.0 / 7.5

7. How I know it is good: testing and validation

Full run on the 50-episode pack (Gemini, rules + LLM):

- 3,727 beats parsed → 39,354 windows → 428 candidates scored → 60 fully scored → 30 delivered.
- 0 of 30 failed final validation; lengths 20.3–57.9 s; the top 10 come from 10 different episodes.
- End refinement moved the end of 23 of the 30 promos to a stronger unresolved cut.
- Rules vs LLM: Spearman correlation 0.46. On cliffhangers they differ by 4+ points 35% of the time, which is why the LLM carries most of the weight.

Robustness checks:

- **Rules-only baseline:** only 4 of its top 10 overlap the LLM's top 10. 7 of the rules' top 10 just end at an episode's end, against 1 of 10 for the LLM run.
- **Ablations:** removing the writer's literal "cliffhanger" annotations keeps 7/10 of the top 10 (moderate dependence). Removing the episode-end prior keeps 5/10.
- **Smoke runs** (3 episodes, live API): 10–12 calls, 74–105 output tokens per candidate, 0 invalid or truncated replies after the fixes.
- **Engineering:** Python regressions, real-browser editor tests, package installation, lint and typing, plus a confidentiality scan (PDFs included) that fails on any 8-word script passage or quoted 4-word line of dialogue in a committable file.

Golden set: not labelled yet, so there is no human-judged accuracy figure. The scorer (precision@10 against `golden.yaml`) is built and tested; the labels must come from an editor, not from me or the AI. The temperature-0 stability re-score exists but was not run on the full pack.

8. Where it picks badly, limitations and next steps

- **Needs backstory:** the tool's own flags mark 23 of 30 promos as needing context for a stranger. This is the biggest weakness.
- **Spoilers:** 4 of 30 reveal an outcome a promo should only tease. **Dead air:** 3 of 30 contain a long silent stretch.
- **Script-only:** slaps and stings are inferred from text, and the judge favours loud melodrama over quiet but effective hooks.
- **Single judge:** one model scores everything. The flags above are the model's own judgement, not human-verified. None of the 40 stitched candidates made the final 30, so multi-moment promos are untested in practice.
- **A reporting bug found late:** the first full run's report counted episode-ending promos before end refinement (14 of 30; the true figure is 1 of 30) and printed a meaningless tokens-per-candidate line. Fixed with tests; the raw outputs were then regenerated from the response cache with the fixed code (4 new LLM calls, identical ranking). The earlier run is kept unedited.

Scale and cost. Candidate generation is pure code: about 13 s for 50 episodes on an 8 GB laptop with no LLM. By construction one show needs about 70 LLM calls for the main ranking (428 candidates in batches of 8, stitched in batches of 5, clarity in batches of 6, 2 rerank passes), at about 2,200 input tokens per scoring call. That is within the 200 requests/day configured for the free key; the cache makes an interrupted run resume without re-billing. At catalogue scale it is a per-show batch job; paid cost is calls × token price, capped by per-run budgets. **Next:** editor-labelled golden set; a one-line show premise in the blind clarity check to cut false "needs context" flags; a stronger judge via `.env`; fit the weights to how posted promos actually perform.

9. AI tools used, and one place they got it wrong

Claude Code built the initial pipeline; Codex assisted with the modular package, FastAPI/React editor, browser tests and final audit. Development history and corrections are logged in `AI_USAGE.md`. **One place it got it wrong:** it matched the model's candidate IDs case-sensitively. Gemini answered `"C1"`, the code expected `c1`, and the always-lowercase mock let all 169 tests pass, but the first real smoke run rejected 11 of 36 answers. Case-insensitive matching plus a regression test fixed it (0 rejections on the next run).